Experiment 01

Advanced SQL Queries

Aim

To study and implement advanced SQL queries using conditional logic, aggregate functions, joins, subqueries, self-joins, and date operations to solve real-world database problems.

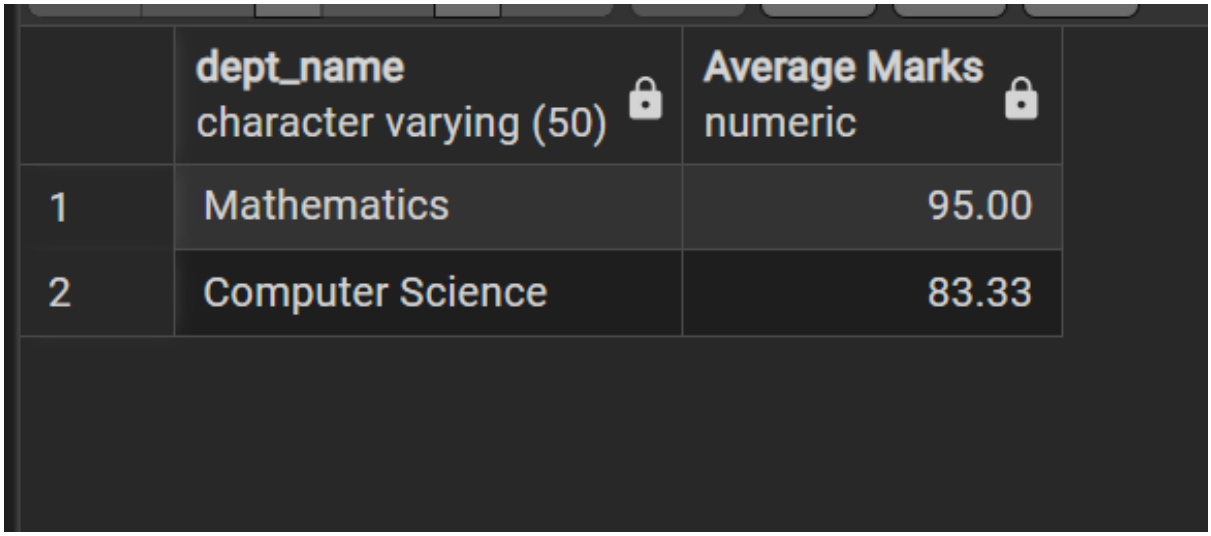
Question 1

A university administration requires an analytical performance report for each academic unit. Write a PostgreSQL solution to calculate the average marks scored by students grouped by their respective departments.

Answer

```sql
SELECT
    d.department_name,
    ROUND(AVG(g.marks),2) AS average_marks
FROM students s
JOIN departments d
ON s.department_id = d.department_id
JOIN grades g
ON s.student_id = g.student_id
JOIN subjects sub
ON g.subject_id = sub.subject_id
GROUP BY d.department_name
ORDER BY average_marks DESC;
```

Output



	dept_name character varying (50)	Average Marks numeric
1	Mathematics	95.00
2	Computer Science	83.33

Question 2

Find the highest paid employee in every department. If multiple employees have the same highest salary, display all of them.

Answer

```sql
SELECT
    d.dept_name,
    e.name,
    e.salary
FROM employee_02 e
JOIN department_02 d
ON e.department_id = d.id
WHERE (e.department_id,e.salary) IN
(
    SELECT
        department_id,
        MAX(salary)
    FROM employee_02
    GROUP BY department_id

)
ORDER BY d.dept_name;
```

Output

	dept_name character varying (50)	name character varying (50)	salary integer
1	IT	JIM	90000
2	IT	MAX	90000
3	SALES	HENRY	80000

Question 3

Part B: Find the second highest earning employee from the same schema.

Answer

```sql
SELECT
    d.dept_name,
    e.name,
    e.salary
FROM employee_02 e
JOIN department_02 d
```

```
ON e.department_id = d.id
WHERE e.salary =
(
    SELECT MAX(salary)
    FROM employee_02
    WHERE department_id = e.department_id
    AND salary <
    (
        SELECT MAX(salary)
        FROM employee_02
        WHERE department_id = e.department_id
    )
)
ORDER BY d.dept_name;
```

Output

	dept_name character varying (50)	name character varying (50)	salary integer
1	IT	JOE	70000
2	SALES	SAM	60000

Question 4

Two legacy HR systems (A and B) have separate records of employee salaries. These records may overlap. Management wants to merge these datasets and identify each unique employee (by EmpID) along with their lowest recorded salary across both systems.

Objective:

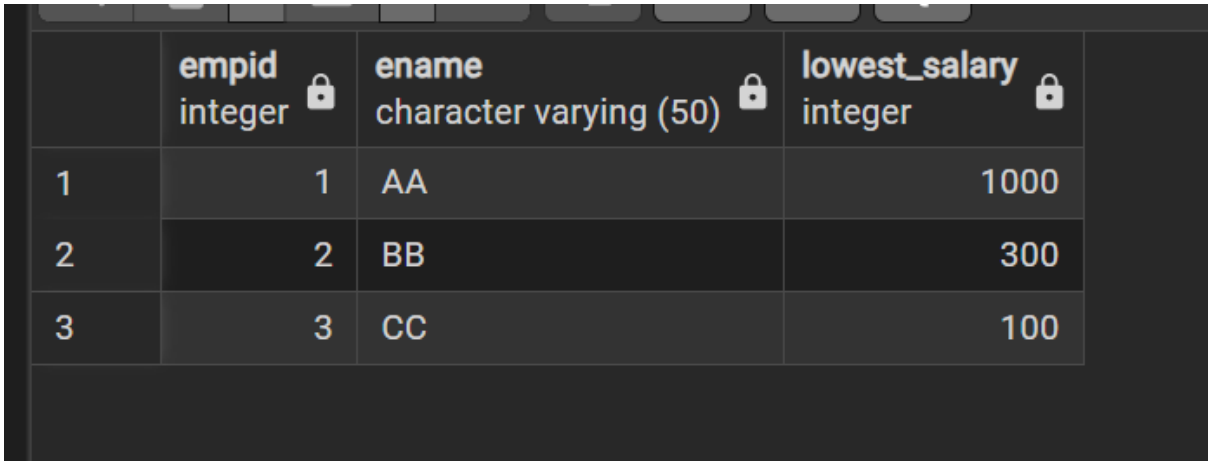
- Combine the records from Table A and Table B.
- Return each EmpID with the corresponding Employee Name and the lowest salary recorded across both tables.

Answer

```
SELECT
    EmpID,
    Ename,
    MIN(Salary) AS Lowest_Salary
FROM
(
    SELECT EmpID, Ename, Salary
    FROM TableA

    UNION ALL

```

```
    SELECT EmpID, Ename, Salary
    FROM TableB
) AS Combined

GROUP BY
    EmpID,
    Ename

ORDER BY
    EmpID;
```

Output



	empid integer	ename character varying (50)	lowest_salary integer
1	1	AA	1000
2	2	BB	300
3	3	CC	100

Learning Outcome

- Learned to use aggregate functions for data analysis.
- Applied JOIN operations on multiple related tables.
- Implemented GROUP BY and AVG functions.
- Retrieved highest and second-highest records using SQL.
- Improved PostgreSQL query writing skills.